SRINIVAS UNIVERSITY

INSTITUTE OF COMPUTER SCIENCE AND INFORMATION SCIENCES

FIRST SEMESTER QUESTION BANK

COURSE CODE: 25MCA12

COURSE NAME: ADVANCED JAVA

CREDITS: 4

Short Answer Questions (7 Marks)

1. Explain JSP technology in detail. Discuss the need for JSP and its advantages over Servlets.?

A: **JSP (JavaServer Pages) Technology**

What is JSP?

JavaServer Pages (JSP) is a **server-side web technology** used to create **dynamic web pages**. It allows developers to embed **Java code inside HTML**, making it easier to generate dynamic content for web applications.

JSP is part of the **Java EE (now Jakarta EE)** platform and runs on a **web container** such as Apache Tomcat.

How JSP Works

1. A client requests a **.jsp** page.
2. The web container translates the JSP into a **Servlet**.

3. The servlet is compiled into a **.class** file.
4. The servlet executes and generates dynamic content.
5. The response is sent back to the client.

✔ This translation happens only once; subsequent requests directly execute the servlet.

Need for JSP

1. Separation of Presentation and Business Logic

- Servlets mix HTML and Java code, making them hard to maintain.
- JSP allows HTML designers and Java developers to work separately.

2. Simplified Web Development

- Writing HTML inside Java code (Servlets) is complex.
- JSP allows Java code inside HTML, which is easier to manage.

3. Faster Development

- JSP reduces development time.
- Less boilerplate code compared to Servlets.

Features of JSP

- **Implicit objects** (`request`, `response`, `session`, etc.)
- **JSP tags** and **Expression Language (EL)**
- **JSTL (JSP Standard Tag Library)**
- Supports MVC architecture

Advantages of JSP over Servlets

1. Easier Page Design

- JSP is HTML-centric.
- Better for UI development.

2. Less Code

- No need to write `println()` statements.
- Cleaner and more readable code.

3. Automatic Servlet Handling

- JSP automatically converts into Servlet.
- No need to manage servlet lifecycle explicitly.

4. Better Maintainability

- Easy to modify UI without touching Java code.
- Improves code readability.

5. Built-in Support for Tags and EL

- Reduces Java code in web pages.
- Makes JSP pages more readable and secure.

6. Faster Development Cycle

- Ideal for rapid application development.

2.Describe the life cycle of a JSP page with neat explanation of each phase.

A: Life Cycle of a JSP Page

The **JSP life cycle** describes the sequence of steps followed by a JSP page from its creation until its destruction. A JSP page is ultimately **translated into a Servlet**, and its life cycle is managed by the **web container** (such as Apache Tomcat).

Phases of JSP Life Cycle

1. Translation Phase

- When a JSP page is requested for the **first time**, the web container translates the JSP file into a **Servlet source file (`.java`)**.
- JSP elements like scriptlets, directives, and expressions are converted into equivalent servlet code.

Purpose:
To convert JSP code into executable Java servlet code.

---

2. Compilation Phase

- The generated servlet source file is compiled into a **Servlet class file (`.class`)**.
- Compilation errors, if any, occur at this stage.

Purpose:
To make the JSP executable by the JVM.

---

3. Loading and Instantiation Phase

- The compiled servlet class is **loaded into memory** by the class loader.
- An instance of the servlet is created.

Purpose:
To prepare the JSP servlet instance for execution.

---

4. Initialization Phase (`jspInit()`)

- The container calls the **`jspInit()`** method.
- This method is called **only once** during the JSP life cycle.
- Used to perform initialization tasks like:
  - Opening database connections
  - Loading configuration data

Purpose:
To initialize resources required by the JSP.

---

5. Request Processing Phase (`_jspService()`)

- For every client request, the **`_jspService()`** method is called.
- This method processes the request and generates the response.
- It handles:
  - Request parameters
  - Business logic
  - Dynamic content generation

Purpose:
To handle client requests and produce responses.

✔ This is the **most frequently executed phase**.

---

6. Destruction Phase (`jspDestroy()`)

- When the JSP page is no longer needed or the server is shut down, the container calls **`jspDestroy()`**.
- Used to release resources like:
  - Closing database connections
  - Freeing memory

Purpose:
To clean up resources before removing the JSP from memory.

JSP Life Cycle Diagram (Text Representation)

```
JSP Page
   ↓
Translation
   ↓
Compilation
   ↓
Class Loading & Instantiation
   ↓
jspInit()
   ↓
_jspService()   (for each request)
   ↓
jspDestroy()
```

3.Explain the processing of a JSP page from client request to response generation.

A: Processing of a JSP Page from Client Request to Response Generation

A **JSP (JavaServer Pages)** page is used to generate dynamic web content. When a client requests a JSP page, the **web container** (such as Apache Tomcat) processes the request through a series of well-defined steps before sending the response back to the client.

---

1. Client Request

- The client (browser) sends an **HTTP request** for a JSP page.
- Example:

```
http://localhost:8080/app/login.jsp
```

---

2. JSP Translation

- If the JSP page is requested **for the first time** (or if it has been modified), the container:
  - Translates the JSP file into a **Servlet source file (`.java`)**
- JSP elements such as:
  - Scriptlets
  - Expressions
  - Directives

  are converted into equivalent servlet code.

---

3. JSP Compilation

- The generated servlet source file is **compiled** into a **Servlet class file (`.class`)**.
- Compilation errors (if any) are detected at this stage.

✔ Translation and compilation happen **only once**, unless the JSP is modified.

---

4. Servlet Loading and Initialization

- The compiled servlet class is **loaded into memory**.
- An instance of the servlet is created.
- The container calls the `jspInit()` method.
- This method is called **only once** and is used for initialization tasks like:
  - Loading configuration data
  - Opening database connections

---

5. Request Processing (`_jspService()` Method)

- For **every client request**, the container invokes the `_jspService()` method.
- This method:
  - Processes request parameters
  - Executes embedded Java code
  - Generates dynamic content
- The output is written to the response object.

✔ This is the **core processing phase**.

---

6. Response Generation

- The generated output (HTML) is sent back to the client as an **HTTP response**.
- The browser renders the response and displays the web page to the user.

Flow Diagram (Text Representation)

```
Client Request
      ↓
JSP Translation
      ↓
JSP Compilation
      ↓
Servlet Loading & jspInit()
      ↓
_jspService()
      ↓
Response Sent to Client
```

4.Explain JSP application design using the MVC architecture with a suitable diagram.

A: JSP Application Design Using MVC Architecture

What is MVC?

MVC (Model–View–Controller) is a **software design pattern** that separates an application into three interconnected components:

- **Model** – Business logic and data
- **View** – Presentation layer (UI)
- **Controller** – Request handling and control logic

In **JSP-based web applications**, MVC helps in building **clean, maintainable, and scalable applications**.

Components of MVC in JSP

1. Model

Role:

- Represents the **business logic** and **data**
- Interacts with databases
- Performs calculations and validations

Examples in JSP applications:

- JavaBeans
- DAO (Data Access Objects)
- Service classes

✔ Independent of presentation and request handling

---

2. View

Role:

- Responsible for **displaying data** to the user
- Contains **no business logic**

Examples:

- JSP pages
- JSTL and Expression Language (EL)

✔ Used only for UI rendering
✔ Fetches data from the Model via the Controller

---

3. Controller

Role:

- Acts as an **intermediary** between Model and View
- Handles **client requests**
- Controls application flow

Examples:

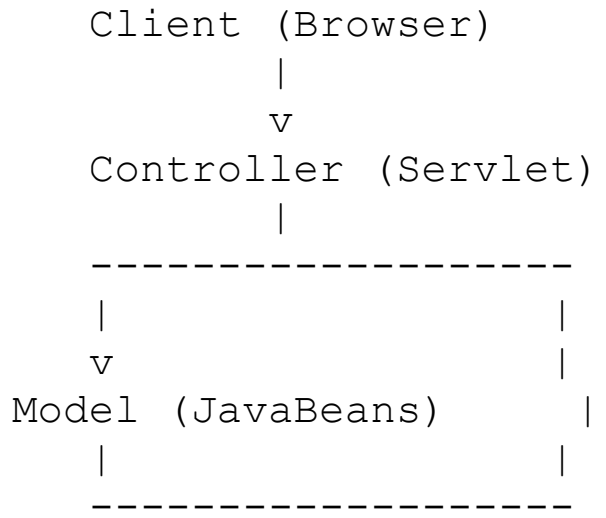
- Servlets (e.g., `LoginServlet`, `EmployeeServlet`)

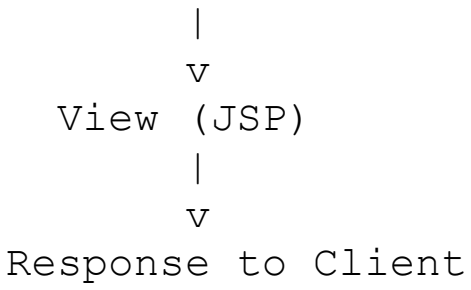
✔ Decides which JSP page to display
✔ Calls Model methods and forwards data to View

Working of MVC in JSP (Step-by-Step)

1. Client sends a request (e.g., login request)
2. **Controller (Servlet)** receives the request
3. Controller invokes **Model** to process business logic
4. Model returns processed data to Controller
5. Controller forwards the data to **View (JSP)**
6. JSP displays the response to the client

MVC Architecture Diagram (Neat Representation)





Advantages of MVC in JSP Applications

1. **Separation of concerns**
2. Better **code maintainability**
3. Easy **team collaboration**
4. Reusable business logic
5. Improved scalability
6. Cleaner and structured application design

5.Describe the different JSP directives and JSP actions with appropriate examples.

A: JSP Directives and JSP Actions

In JSP, **directives** and **actions** are used to control page behavior and to perform specific tasks at runtime. Directives provide instructions to the JSP container during the **translation phase**, while actions are executed during the **request processing phase**.

Part A: JSP Directives

JSP directives give **global information** to the JSP container about the page.

1. `page` Directive

Purpose:
Defines page-specific settings such as language, content type, imports, and error handling.

Syntax:

```
<%@ page language="java" contentType="text/html"
import="java.util.*" %>
```

Common Attributes:

- `language`
- `contentType`
- `import`
- `session`
- `errorPage`
- `isErrorPage`

Example:

```
<%@ page import="java.sql.*" %>
```

---

2. `include` Directive

Purpose:
Includes the content of another file **at translation time**.

Syntax:

```
<%@ include file="header.jsp" %>
```

Example Use:
Including header or footer files.

✔ Content becomes part of the JSP at compile time.

---

3. `taglib` Directive

Purpose:
Used to define custom tag libraries such as **JSTL**.

Syntax:

```
<%@ taglib
uri="http://java.sun.com/jsp/jstl/core"
prefix="c" %>
```

Example:

```
<c:out value="${username}" />
```

Part B: JSP Actions

JSP actions are **XML-based tags** that perform tasks at **runtime**.

---

1. `<jsp:include>`

Purpose:
Includes another resource **at request time**.

Syntax:

```
<jsp:include page="menu.jsp" />
```

Difference from directive include:

- Dynamic inclusion
- Content can change per request

---

2. `<jsp:forward>`

Purpose:
Forwards a request to another resource.

Syntax:

```
<jsp:forward page="home.jsp" />
```

✔ Used for navigation and flow control.

---

3. `<jsp:useBean>`

Purpose:
Creates or locates a JavaBean.

Syntax:

```
<jsp:useBean id="user" class="com.model.User"
scope="session" />
```

---

4. `<jsp:setProperty>`

Purpose:
Sets a property value of a JavaBean.

Syntax:

```
<jsp:setProperty name="user" property="name"
value="Ravi" />
```

---

5. `<jsp:getProperty>`

Purpose:
Retrieves a property value from a JavaBean.

Syntax:

```
<jsp:getProperty name="user" property="name" />
```

6.Explain JSP implicit objects and their role in form processing and session handling.

A: JSP Implicit Objects:

JSP implicit objects are predefined objects created automatically by the JSP container. These objects are available to the developer **without explicit declaration** and are used to handle **requests, responses, sessions, application data, and output**.

There are **9 implicit objects** in JSP.

List of JSP Implicit Objects

1. `request`
2. `response`
3. `session`
4. `application`
5. `out`
6. `config`
7. `pageContext`
8. `page`
9. `exception`

Role of Implicit Objects in Form Processing

1. `request` Object

Purpose:

- Collects data sent from HTML forms
- Stores request-scope attributes

Example:

```
<%
String uname = request.getParameter("username");
String pwd = request.getParameter("password");
%>
```

✔ Used to read form inputs using `getParameter()`
✔ Supports both GET and POST methods

---

2. `response` Object

Purpose:

- Sends output back to the client
- Controls redirection and content type

Example:

```
response.sendRedirect("welcome.jsp");
```

✔ Redirects user after form submission

---

3. `out` Object

Purpose:

- Writes data to the response body

Example:

```
out.println("Welcome " + uname);
```

✔ Displays processed form data

Role of Implicit Objects in Session Handling

4. `session` Object

Purpose:

- Maintains user state across multiple requests
- Stores user-specific data

Example:

```
session.setAttribute("user", uname);
```

Retrieve:

```
String user = (String)
session.getAttribute("user");
```

✔ Used for login tracking, shopping carts, and personalization

---

5. `application` Object

Purpose:

- Shares data across all users and sessions

- Application-level storage

Example:

```
application.setAttribute("visitorCount", 100);
```

✔ Used for global counters and configuration data

---

6. `pageContext` Object

Purpose:

- Provides access to all scopes (page, request, session, application)
- Manages page-related attributes

Example:

```
pageContext.setAttribute("msg", "Hello",
PageContext.REQUEST_SCOPE);
```

✔ Acts as a central access point

Other Implicit Objects (Brief)

7. `config`

- Accesses servlet configuration parameters

8. `page`

- Refers to the current JSP page instance

9. `exception`

- Used in error pages to handle exceptions

7.Explain session tracking in JSP using sessions and cookies.

A: Session Tracking in JSP:

What is Session Tracking?

Session tracking is a mechanism used to **maintain user state** across multiple HTTP requests. Since **HTTP is stateless**, JSP uses techniques like **sessions and cookies** to identify and track a user during a visit to a web application.

Session Tracking Using Sessions

JSP Session Object

JSP provides the **`session` implicit object**, which represents an instance of `HttpSession`.

How It Works

- A session is created when a user accesses a JSP page
- The server assigns a **unique session ID**
- The session ID is stored on the client (usually in a cookie)

Storing Data in Session

```
session.setAttribute("username", "Ravi");
```

Retrieving Data from Session

```
String user = (String)
session.getAttribute("username");
```

Removing Session Data

```
session.removeAttribute("username");
```

Invalidating a Session

```
session.invalidate();
```

Advantages

- Secure (data stored on server)
- Easy to use
- Supports complex objects

Disadvantages

- Uses server memory
- Not ideal for very large user bases

Session Tracking Using Cookies

What Are Cookies?

Cookies are **small text files** stored on the client's browser. They are used to store user-related data and sent back to the server with each request.

Creating a Cookie in JSP

```
Cookie c = new Cookie("username", "Ravi");
response.addCookie(c);
```

Reading Cookies

```
Cookie[] cookies = request.getCookies();
for(Cookie ck : cookies) {
```

```
    out.println(ck.getName() + " : " +
ck.getValue());
}
```

Types of Cookies

1. **Session Cookies** – Deleted when browser closes
2. **Persistent Cookies** – Stored with expiry time

Advantages

- Reduces server load
- Simple and lightweight

Disadvantages

- Less secure
- Limited storage size

8.Describe JSP database access using JDBC. Explain the steps involved with example.

A: JSP Database Access Using JDBC

JDBC (Java Database Connectivity) is an API that allows Java applications, including JSP pages, to **connect and interact with relational databases** such as MySQL, Oracle, or SQL Server.

In JSP, JDBC enables developers to perform **CRUD operations** (Create, Read, Update, Delete) from web pages.

Steps Involved in JSP Database Access:

1. Load JDBC Driver

- Load the database driver class using `Class.forName()`.
- Required to enable communication between Java/JSP and the database.

Example:

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

---

2. Establish Database Connection

- Use `DriverManager.getConnection()` to connect to the database.
- Provide **URL, username, and password**.

Example:

```
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/companydb",
"root", "password");
```

---

3. Create Statement

- Create a `Statement` or `PreparedStatement` object to execute SQL queries.

Example:

```
Statement stmt = con.createStatement();
```

---

4. Execute SQL Query

- Use the `executeQuery()` method for SELECT queries
- Use `executeUpdate()` method for INSERT, UPDATE, DELETE

Example (SELECT):

```
ResultSet rs = stmt.executeQuery("SELECT * FROM
employees");
```

---

5. Process Result Set

- Iterate over the `ResultSet` to read data returned from the database.
- Display results using JSP `out` object.

Example:

```
while(rs.next()) {
    out.println("Employee Name: " +
rs.getString("name") + "<br>");
}
```

---

6. Close Connections

- Always close **ResultSet, Statement, and Connection** to free resources.

Example:

```
rs.close();
stmt.close();
con.close();
```

Complete JSP Example

```
<%@ page import="java.sql.*" %>
<html>
<body>
<h2>Employee List</h2>
<%
try {
    // 1. Load Driver
    Class.forName("com.mysql.cj.jdbc.Driver");

    // 2. Establish Connection
    Connection con =
DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/companydb",
"root", "password");

    // 3. Create Statement
    Statement stmt = con.createStatement();

    // 4. Execute Query
    ResultSet rs = stmt.executeQuery("SELECT *
FROM employees");

    // 5. Process ResultSet
    while(rs.next()) {
        out.println("ID: " + rs.getInt("empId")
+
                    ", Name: " +
rs.getString("name") +
                    ", Dept: " +
rs.getString("department") + "<br>");
    }

    // 6. Close Connection
    rs.close();
    stmt.close();
```

```
    con.close();
} catch(Exception e) {
    out.println("Error: " + e.getMessage());
}
%>
</body>
</html>
```

9.Explain JSP Standard Tag Library (JSTL), Expression Language (EL), and exception handling in JSP.

A: JSP Standard Tag Library (JSTL), Expression Language (EL), and Exception Handling

JSP provides several mechanisms to **simplify web development** and reduce the amount of Java code embedded in pages. These include **JSTL**, **EL**, and **exception handling**.

---

1. JSP Standard Tag Library (JSTL)

Definition:
JSTL is a **collection of custom JSP tags** that encapsulate common tasks like iteration, conditional logic, database access, and internationalization.

Purpose:

- Reduce the use of scriptlets in JSP pages
- Make JSP pages more readable and maintainable

Core JSTL Tag Categories:

Tag Category	Purpose
Core (`<c:...>`)	Conditional statements, loops, URL redirects, and variable manipulation
Formatting (`<fmt:...>`)	Internationalization and formatting of numbers/dates
SQL (`<sql:...>`)	Execute queries and update database (not recommended for production)
XML (`<x:...>`)	Processing XML data
Functions (`<fn:...>`)	String manipulation

Example (Core JSTL Loop):

```
<%@ taglib
uri="http://java.sun.com/jsp/jstl/core"
prefix="c" %>
<c:forEach var="emp" items="${employees}">
    Name: ${emp.name} <br>
</c:forEach>
```

✔ This iterates over a collection without using Java scriptlets.

---

2. Expression Language (EL)

Definition:
EL is a simple language used to **access data stored in JSP scopes** like request, session, application, or page.

Purpose:

- Simplifies access to JavaBeans and other objects
- Eliminates complex Java scriptlets

Syntax:

```
${expression}
```

Examples:

```
${user.name}          <!-- Access property
'name' of bean 'user' -->
${sessionScope.username}  <!-- Access session
attribute -->
${param.email}        <!-- Access form parameter
'email' -->
```

✔ EL works with JSP implicit objects and scopes automatically.

---

3. Exception Handling in JSP

Definition:
JSP provides a way to **handle runtime errors** using the `errorPage` directive and `isErrorPage` attribute.

Steps:

1. **Define an Error Page**

```
<%@ page isErrorPage="true" %>
<html>
<body>
<h2>Error Occurred</h2>
<p>${exception}</p>
</body>
</html>
```

2. **Set Error Page in Main JSP**

```
<%@ page errorPage="error.jsp" %>
```

Explanation:

- If a runtime exception occurs in the main JSP, it is automatically redirected to the **error page**.
- The `exception` implicit object contains the error details.

Advantages:

- Centralized error handling
- Prevents showing stack traces to users
- Improves maintainability